

CapFinLoan - Low Level Design (LLD)

1. Architectural Style

The CapFinLoan application is built using a **Microservices Architecture**. Each logical domain is isolated into its own independent executable and database, ensuring strict bounded contexts and high availability.

Each individual Microservice follows **Clean Architecture (Onion Architecture)** principles, enforcing a strict dependency rule where inner layers contain business logic and outer layers contain infrastructure details.

2. Microservice Structure Layout

Each microservice is divided into 5 standard .NET 10 libraries/projects:

- **.Domain Layer:**
 - Contains all Enterprise business logic and Entities (e.g., `User.cs`, `LoanApplication.cs`).
 - Strict absence of dependencies; does not rely on EF Core or web primitives.
- **.Application Layer:**
 - Contains Use Cases and specific Application logic.
 - Houses **Services**, **DTOs** (Data Transfer Objects), **Interfaces** (Repository contracts like `IApplicationRepository`), and **Custom Exceptions** (e.g., `ApplicationValidationException`).
- **.Persistence Layer:**
 - Implements the Data Access layer.
 - Contains EF Core `DbContext` configurations and actual realizations of the Repository interfaces.
 - Manages EF Core Code-First Migrations.
- **.Infrastructure Layer (Optional per service):**
 - Defines external implementations such as file storage integrations (for Documents) or specific notification libraries.
- **.API Layer:**
 - Presentation layer consisting of ASP.NET Core Controllers and Middlewares.
 - Hosts the `Program.cs` composition root where Dependency Injection (DI) constraints are mapped.
 - RabbitMQ messaging event bus integrations (consumers/publishers).

3. Data Storage & Schema Configuration

- **System:** Microsoft SQL Server 2022
- **ORM:** Entity Framework Core 10 (EF Core) + Code First Approach.
- **Database Isolation:** The system implements "Database per Service" to secure autonomy.
- `CapFinLoanAuth`
- `CapFinLoanApplication`
- `CapFinLoanDocument`
- `CapFinLoanAdmin`
- *Entity Configurations:* Instead of cluttering domain entities with `[Table]` attributes everywhere, standard `IEntityTypeConfiguration<T>` implementations or Data Annotations dictate schema logic.

4. Specific Design Patterns

Repository Pattern

Data access logic is completely abstracted behind `IRepository` interfaces inside the Application layer. The `.Persistence` layer provides concrete logic, allowing seamless substitutions of Database providers for unit tests.

Saga Pattern (Distributed Transactions)

Since applications span multiple services (`ApplicationService`, `AdminService`), distributed transactions are implemented via Sagas. For example:

`ApplicationSubmissionSagaCoordinator` initiates the saga when a loan is submitted, tracks its state through the event-bus across Admin reviews, and either completes or aggressively compensates (reverts to 'PENDING') on failure.

Data Transfer Object (DTO) Pattern

Enforces that raw Entities are never exposed through API controllers. Transformation is done explicitly at the Service layer ensuring clients only send/receive exactly what they need.

Publish-Subscribe (Event Bus)

Inter-service communication is entirely asynchronous (choreography) leveraging **RabbitMQ**. Services dispatch domain events out to the exchange (`capfinloan.events`). E.g., `document.uploaded` is fired by `DocumentService` and consumed downstream by `AdminService`.

5. Centralized Error Handling

To standardize the JSON error payloads served to the frontend UI, custom structured exceptions are thrown at the Application Service layer (e.g., `throw new ApplicationValidationException("Income too low")`). An `ExceptionHandlerMiddleware` inside each API catches these faults globally and maps them automatically to appropriate HTTP 4xx or 5xx codes.

6. Security (Authentication & Authorization)

- Authentication relies uniformly on **JWT (JSON Web Tokens)**.
- Tokens are issued uniquely through the `AuthService` upon validation of Email OTPs or Google OAuth tokens.
- All microservice controllers inject the standard `[Authorize]` attribute to enforce token presence.
- Roles are extracted automatically from the Claims Principal (`ClaimTypes.Role`) to gate elevated calls with `[Authorize(Roles="ADMIN")]`.

7. Ocelot API Gateway

Direct port consumption of downstream microservices (Ports 5000–5003) is completely blocked in the deployment environment. Standardly, the frontend hits **Ocelot (Port 5021)**, which reverse proxies and reroutes path segments (`/gateway/applications/...`) into internal target networks. The gateway centralizes CORS headers protecting the inner architecture.